Министерство науки и высшего образования РФ

Пензенский государственный университет

Кафедра «Вычислительная техника»

Отчет

по лабораторной работе №3

по дисциплине «Программирование на языке Java»

на тему: «Обработка исключительных ситуаций»

Вариант 3

Выполнили студенты группы 23ВВВ3:

Балаев Г. С.

Саранцев. Е. А.

Приняли:

Юрова О. В.

Карамышева Н. С.

Пенза, 2026

Цель работы

изучить механизм обработки исключительных ситуаций.

Лабораторное задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав проверку вводимых данных с использованием механизма исключений. Необходимо создать свой класс, унаследованный от класса Exception, и генерировать исключение, если возникает попытка создать экземпляр класса RecIntegral со значениями, не являющимися числами в диапазоне от 0,000001 до 1000000. В качестве обработки исключения необходимо выводить диалог, содержащий предупреждение о некорректности введенных данных. Оформление лабораторной работы должно быть выполнено в соответствии с требованиями, приведенными в Приложении 2.

Ход работы:

1. Открыли прошлый проект в IDE JetBrains и создали новый класс InvalidDataException.

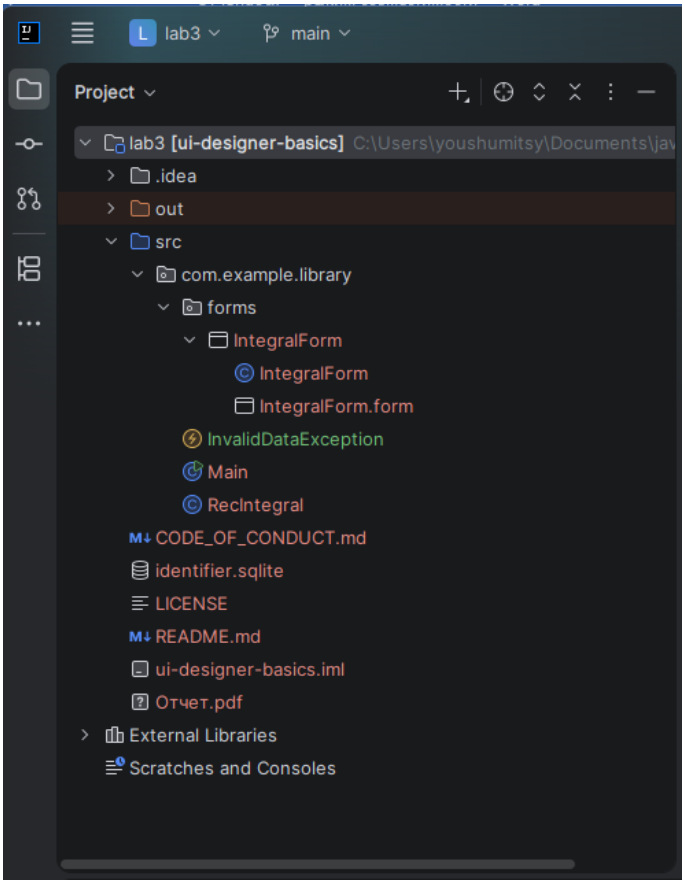

Рисунок 1 – Проект.

2. Добавили обработку исключений NumberFormatException, InvalidDataException, ArrayIndexOutOfBoundsException.

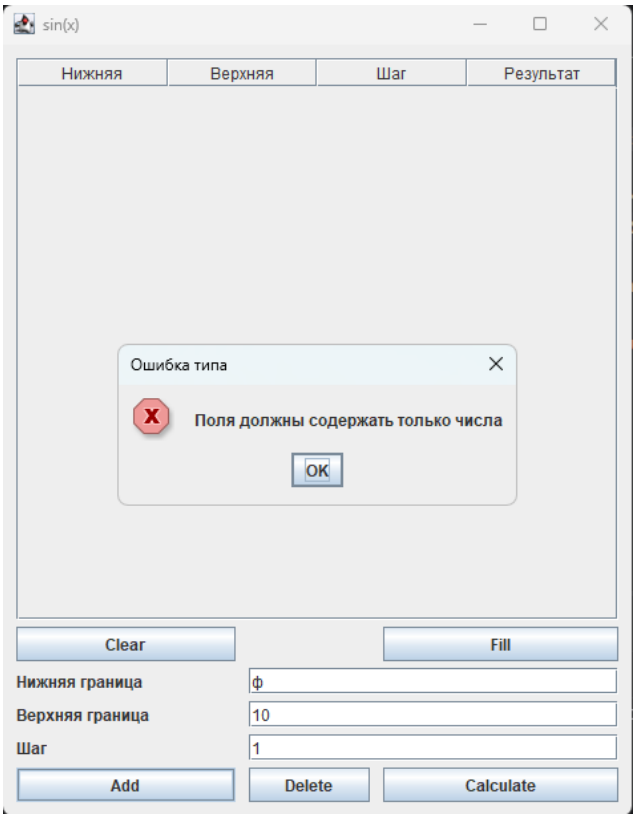

Рисунок 2 – Исключение NumberFormatException.

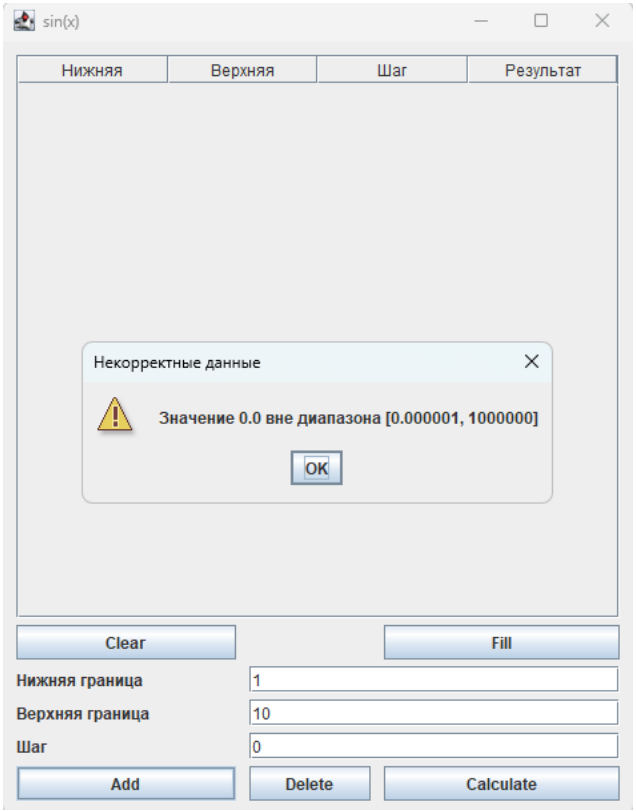

Рисунок 3 – Исключение InvalidDataException.

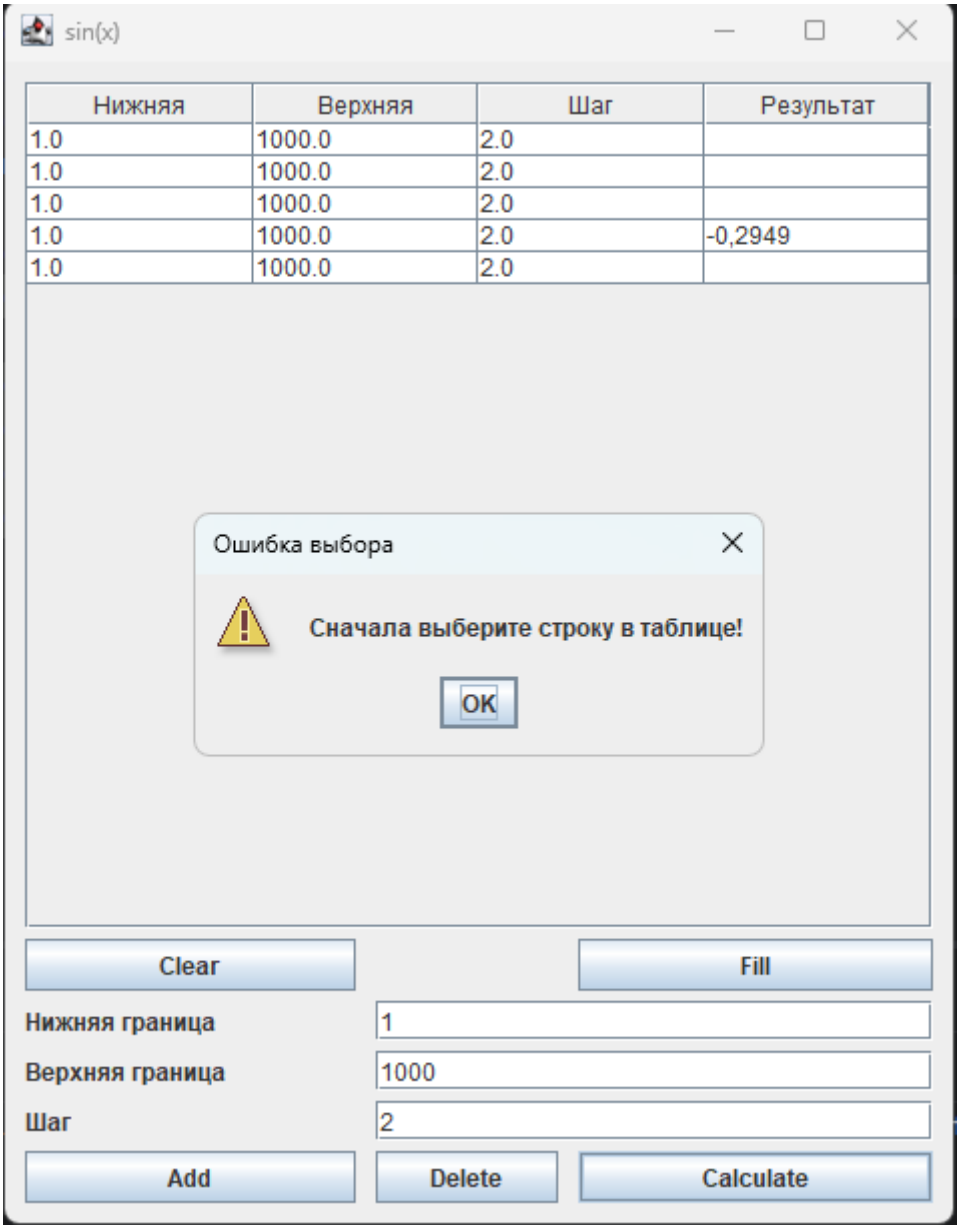

Рисунок 4 – Исключение ArrayIndexOutOfBoundsException.

Результат работы программы:

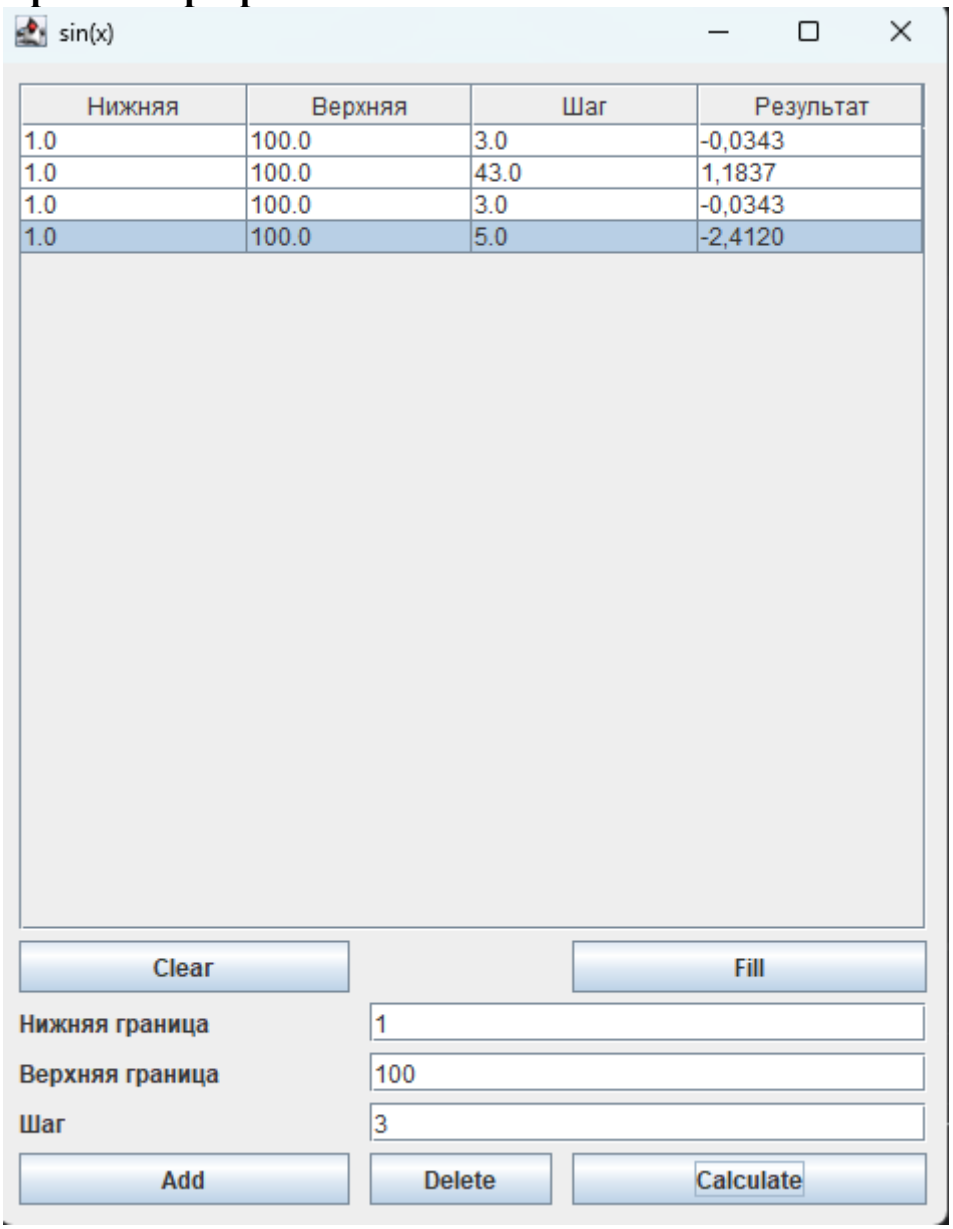

Рисунок 5 – Результат работы программы.

Вывод:

В ходе данной лабораторной работы мы научились работе с механизмом обработки исключительных ситуаций.

Листинг: Main.java:

```
package com.example.library;

import javax.swing.*;
import com.example.library.forms.IntegralForm;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                JFrame frame = new JFrame("sin(x)");

                IntegralForm form = new IntegralForm();
                frame.setContentPane(form.getRootPanel());

                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
                frame.pack();
                frame.setLocationRelativeTo(null);
                frame.setVisible(true);
            }
        });
    }
}
```

IntegralForm.java:

```
package com.example.library.forms;

import com.example.library.InvalidDataException;
import com.example.library.RecIntegral;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.util.ArrayList;
import java.util.List;

public class IntegralForm {
    private JPanel rootPanel;
    private JTextField fieldLowerBound;
    private JTextField fieldUpperBound;
    private JTextField fieldStep;
    private JButton btnAdd;
    private JButton btnDelete;
    private JButton btnCalculate;
    private JTable table1;
    private boolean clrClicked = false;
    private JButton btnClear;
    private JButton btnFill;

    private DefaultTableModel model;
    private List<RecIntegral> listRecords = new ArrayList<>();

    public IntegralForm() {
        String[] columns = {"Нижняя", "Верхняя", "Шаг", "Результат"};
        model = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return column != 3;
            }

            @Override
            public void setValueAt(Object aValue, int row, int column) {
                if (column == 3) {
                    super.setValueAt(aValue, row, column);
```

```
                    return;
                }
                try {
                    double val =
Double.parseDouble(aValue.toString().replace(',', '.'));
                    if (val < 0.000001 || val > 1000000) {
                        throw new InvalidDataException("Число " + val + " вне
диапазона!");
                    }
                    super.setValueAt(aValue, row, column);

                    if (row >= 0 && row < listRecords.size()) {
                        RecIntegral rec = listRecords.get(row);
                        if (column == 0) rec.setLowerBound(val);
                        else if (column == 1) rec.setUpperBound(val);
                        else if (column == 2) rec.setStep(val);

                        rec.setResult(0);
                        super.setValueAt("", row, 3);
                    }
                } catch (NumberFormatException e) {
                    JOptionPane.showMessageDialog(rootPanel, "Введите корректное
число", "Ошибка ввода", JOptionPane.ERROR_MESSAGE);
                } catch (InvalidDataException e) {
                    JOptionPane.showMessageDialog(rootPanel, e.getMessage(),
"Ошибка диапазона", JOptionPane.WARNING_MESSAGE);
                }
            }
        };
        table1.setModel(model);

        btnAdd.addActionListener(e -> {
            try {
                double low = Double.parseDouble(fieldLowerBound.getText());
                double high = Double.parseDouble(fieldUpperBound.getText());
                double step = Double.parseDouble(fieldStep.getText());
                RecIntegral record = new RecIntegral(low, high, step, 0);

                listRecords.add(record);
                model.addRow(new Object[]{low, high, step, ""});

            } catch (NumberFormatException ex) {
                JOptionPane.showMessageDialog(rootPanel, "Введите числа!",
"Ошибка типа", JOptionPane.ERROR_MESSAGE);
            } catch (InvalidDataException ex) {
                JOptionPane.showMessageDialog(rootPanel, ex.getMessage(),
"Некорректные данные", JOptionPane.WARNING_MESSAGE);
            }
        });

        btnCalculate.addActionListener(e -> {
            try {
                int row = table1.getSelectedRow();
                if (row == -1) {
                    throw new ArrayIndexOutOfBoundsException("Строка не
выбрана");
                }

                double left = Double.parseDouble(model.getValueAt(row,
0).toString());
                double right = Double.parseDouble(model.getValueAt(row,
1).toString());
                double step = Double.parseDouble(model.getValueAt(row,
2).toString());

                double res = RecIntegral.calculate(left, right, step);
```

```
                RecIntegral rec = listRecords.get(row);
                rec.setResult(res);
                model.setValueAt(String.format(java.util.Locale.US, "%.4f",
res), row, 3);

            } catch (ArrayIndexOutOfBoundsException ex) {
                JOptionPane.showMessageDialog(rootPanel, "Сначала выберите
строку в таблице!", "Ошибка выбора", JOptionPane.WARNING_MESSAGE);
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(rootPanel, "Ошибка: " +
ex.getMessage());
            }
        });

        btnDelete.addActionListener(e -> {
            try {
                int row = table1.getSelectedRow();
                if (row == -1) {
                    throw new ArrayIndexOutOfBoundsException();
                }
                listRecords.remove(row);
                model.removeRow(row);

            } catch (ArrayIndexOutOfBoundsException ex) {
                JOptionPane.showMessageDialog(rootPanel, "Нечего удалять: строка
не выбрана!", "Ошибка удаления", JOptionPane.WARNING_MESSAGE);
            }
        });
        btnClear.addActionListener(e -> {
            clrClicked = true;
            model.setRowCount(0);
        });

        btnFill.addActionListener(e -> {
            clrClicked = false;
            model.setRowCount(0);
            for (RecIntegral rec : listRecords) {
                String resStr = (rec.getResult() == 0) ? "" :
String.format("%.4f", rec.getResult());
                model.addRow(new Object[]{rec.getLowerBound(),
rec.getUpperBound(), rec.getStep(), resStr});
            }
        });
    }

    public JPanel getRootPanel() {
        return rootPanel;
    }
}
```

RecIntegral.java:

```
package com.example.library;

import static java.lang.Math.sin;

public class RecIntegral {
    private double lowerBound;
    private double upperBound;
    private double step;
    private double result;


    public RecIntegral(double lowerBound, double upperBound, double step,
double result) throws InvalidDataException {
        checkValue(lowerBound);
        checkValue(upperBound);
        checkValue(step);

        this.lowerBound = lowerBound;
        this.upperBound = upperBound;
        this.step = step;
        this.result = result;
    }

    private void checkValue(double val) throws InvalidDataException {
        if (val < 0.000001 || val > 1000000) {
            throw new InvalidDataException("Значение " + val + " вне
диапазона [0.000001, 1000000]");
        }
    }

    public double getLowerBound() { return lowerBound; }
    public void setLowerBound(double lowerBound) { this.lowerBound =
lowerBound; }

    public double getUpperBound() { return upperBound; }
    public void setUpperBound(double upperBound) { this.upperBound =
upperBound; }

    public double getStep() { return step; }
    public void setStep(double step) { this.step = step; }

    public double getResult() { return result; }
    public void setResult(double result) { this.result = result; }

    private static double f(double x) {
        return sin(x);
    }

    public static double calculate(double left, double right, double
step) {
        if (step <= 0) return 0;
        int n = (int)((right - left) / step);
        if (n <= 0) return 0;

        double result = 0;
        for (int i = 0; i < n; i++) {
            double x0 = left + i * step;
            double x1 = x0 + step;
            if (x1 > right) x1 = right;
```

```
            result += (f(x0) + f(x1)) * (x1 - x0) / 2;
        }
        return result;
    }
}
```

InvalidDataException.java:

```
package com.example.library;


public class InvalidDataException extends Exception {
    public InvalidDataException(String message) {
        super(message);
    }
}
```